REPORTE DE INVESTIGACIÓN: LOCUST

Herramientas de Pruebas de Carga y Estrés para APIs

Asignatura:	Programación IV
Profesor:	Diego Alberto Ayala Vázquez
Alumna:	Patricia Jaime
Fecha:	14 de Octubre de 2026
Repositorio GitHub:	`https://github.com/patriciajaime/tienda-fastapi-frontend`

1. ¿Qué es Locust?

Locust es una herramienta *open-source* de pruebas de carga y rendimiento diseñada para servicios web y APIs distribuidas. A diferencia de otras herramientas que requieren interfaces complejas o archivos XML, Locust permite definir el comportamiento de los usuarios concurrentes mediante código **Python nativo y limpio**.

Está construida sobre eventos asíncronos (*coroutines*), lo que le permite simular miles de usuarios simultáneos consumiendo muy pocos recursos del sistema.

2. ¿Qué hace Locust?

Locust ejecuta simulación de tráfico web y métricas en tiempo real mediante las siguientes capacidades principales:

1. **Simulación de Usuarios Virtuales:** Genera múltiples usuarios concurrentes que interactúan con los *endpoints* HTTP (`GET`, `POST`, `DELETE`) respetando tiempos de pausa entre solicitudes.
2. **Medición de Métricas Clave:**
   - **RPS (Requests Per Second):** Número total de peticiones procesadas exitosamente por segundo.
   - **Latencia / Tiempos de Respuesta:** Promedios, mínimos, máximos y percentiles en milisegundos.
   - **Tasa de Error:** Porcentaje de peticiones fallidas (códigos `4xx` y `5xx`).
3. **Panel Gráfico en Tiempo Real:** Proporciona una interfaz web interactiva para ajustar la cantidad de usuarios y observar gráficas de rendimiento en vivo.

3. ¿Para qué nos sirve en nuestro proyecto de la tienda?

En el contexto de nuestra API en **FastAPI con el Middleware de Rate Limiting**, Locust nos sirve para:

- **Validar el Rate Limiting Automáticamente:** En lugar de presionar `Cmd + R` de forma manual en el navegador, Locust puede enviar cargas exactas (por ejemplo, 10, 50 o 100 peticiones por segundo) para comprobar que el middleware responda con código **HTTP 429 Too Many Requests** de forma consistente.

- **Verificar la Concurrencia con `asyncio.Lock`:** Permite someter al backend a peticiones simultáneas de consulta y creación de productos, asegurando que el cerrojo asíncrono evite condiciones de carrera.
- **Determinar el Punto de Quiebre:** Ayuda a conocer la capacidad máxima de solicitudes por segundo que puede soportar nuestro servidor antes de presentar retrasos o errores.

4. Ejemplo Básico (`locustfile.py`)

```python
from locust import HttpUser, task, between

class UsuarioTienda(HttpUser):
    wait_time = between(1, 2)

    @task(3)
    def consultar_productos(self):
        self.client.get("/productos")

    @task(1)
    def agregar_producto(self):
        self.client.post("/productos", json={
            "id": 99,
            "nombre": "Producto Pruebas",
            "precio": 99.99
        })
```

Listing 1: Script de prueba para la API de la tienda

5. Conclusión

Puedo aplicar Locust a mi tienda creando un pequeño archivo de pruebas que simule a muchos clientes entrando al mismo tiempo a mi página. Al ejecutar esta prueba, el programa empezará a consultar la lista de productos y a intentar agregar nuevos artículos de forma automática. De esta manera, no tendré que estar recargando la página manualmente desde mi teclado, sino que veré en una pantalla con gráficas cómo responde mi sistema, comprobando si el límite que le puse funciona correctamente y cuántas visitas seguidas puede resistir mi tienda antes de volverse lenta.